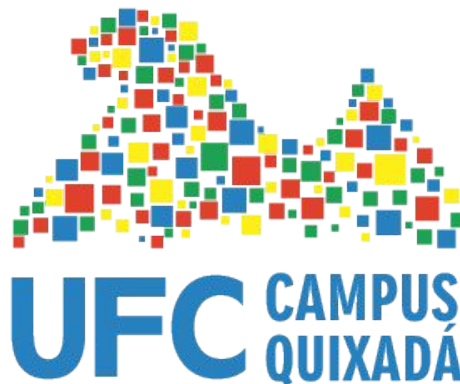


Gerência de Configuração

GC

Sidarta Carvalho



Git Avançado



01. Git Avançado

Domínio de comandos essenciais e resolução de problemas no fluxo de trabalho local.

- Resolução de conflitos
- Descarte com restore
- Estados com reset & revert
- Navegação & Diff



02. Releases

Estratégias de versionamento e preparação de pacotes de software para entrega estável.

- Gerenciamento de Releases
- Marcação de marcos com Tags
- Padrões de Nomenclatura
- Isolamento de features/hotfixes



03. Atividade

Prática intensiva hands-on simulando cenários reais e de crise para consolidação.

- Resolução de crises locais
- Simulação de builds quebrados
- Higienização de commits
- Auditoria de histórico

Git Avançado: Engenharia de Sobrevivência, Workflows e DevOps

Do controle de crises no terminal à automação de pipelines em nuvem.

FEATURE BRANCH (DEV)

git checkout -b feature/new-tool

COMMIT: a1b2c3d

STAGING BRANCH (QA)

git checkout staging

COMMIT: e4f5g6h

HOTFIX BRANCH (CRITICAL)

git checkout -b hotfix/urgent-patch

COMMIT: i7j8k9l

RELEASE v2.0 (STABLE)

git tag v2.0

MAIN BRANCH (PRODUCTION)

git merge --no-ff
git push origin main

BUILD & TEST

docker build
npm test

DEPLOYMENT (AUTOMATED)

kubectl apply -f

ITERATE & PLAN

jira sync

MONITOR & FEEDBACK

datadog metrics

STATUS: OPERATIONAL

Da "Síndrome do Arquivo Final" ao Controle Absoluto

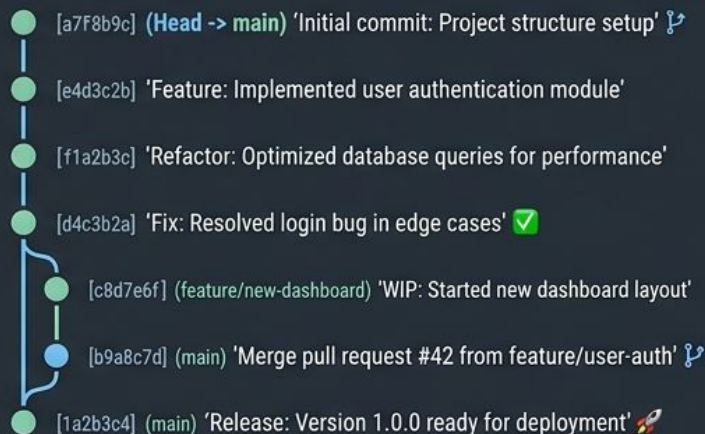
Caos Colaborativo

O medo contínuo de fazer uma grande mudança, quebrar o código e não ter como voltar atrás.

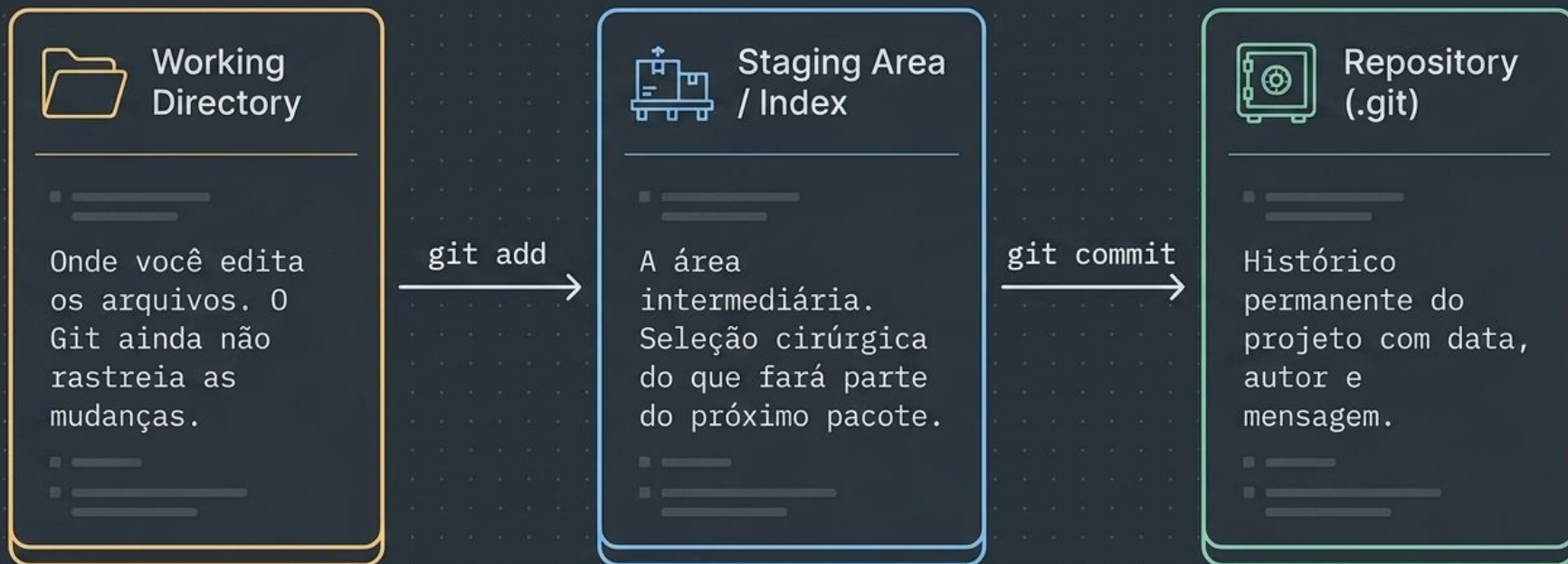


Engenharia de Sobrevivência

O Git não é burocracia. É a rede de segurança distribuída que garante liberdade para criar e falhar sem consequências fatais.



A Arquitetura da Rede de Segurança



Módulo 1: Gestão de Crises e Desfazer Erros

No desenvolvimento de software corporativo, o erro não é uma possibilidade; é uma certeza. A diferença entre um desastre e um contratempo é saber exatamente em qual camada o erro está alojado e qual comando cirúrgico utilizar para neutralizá-lo.



Protocolo de Resposta a Crises: Onde está o erro?



A Máquina do Tempo Local: Níveis de Destruição

`git reset --soft`



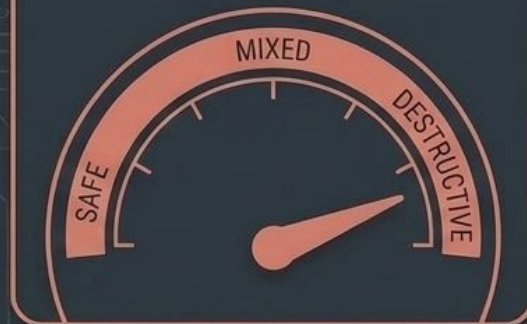
Desfaz o commit, mas mantém todas as alterações na Staging Area. Seguro para juntar pequenos commits (squash).

`git reset --mixed` (Padrão)



Desfaz o commit e tira tudo da Staging Area. O código volta para o Working Directory, pronto para ser revisado.

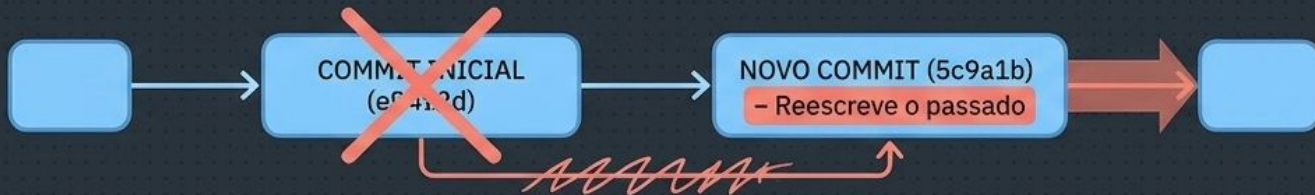
`git reset --hard`



Comando destrutivo. Desfaz o commit e aniquila as alterações do Working Directory. Não há como recuperar.

A Regra de Ouro: Local vs. Remoto

Reescrita de Histórico Permitida – Máquina Local



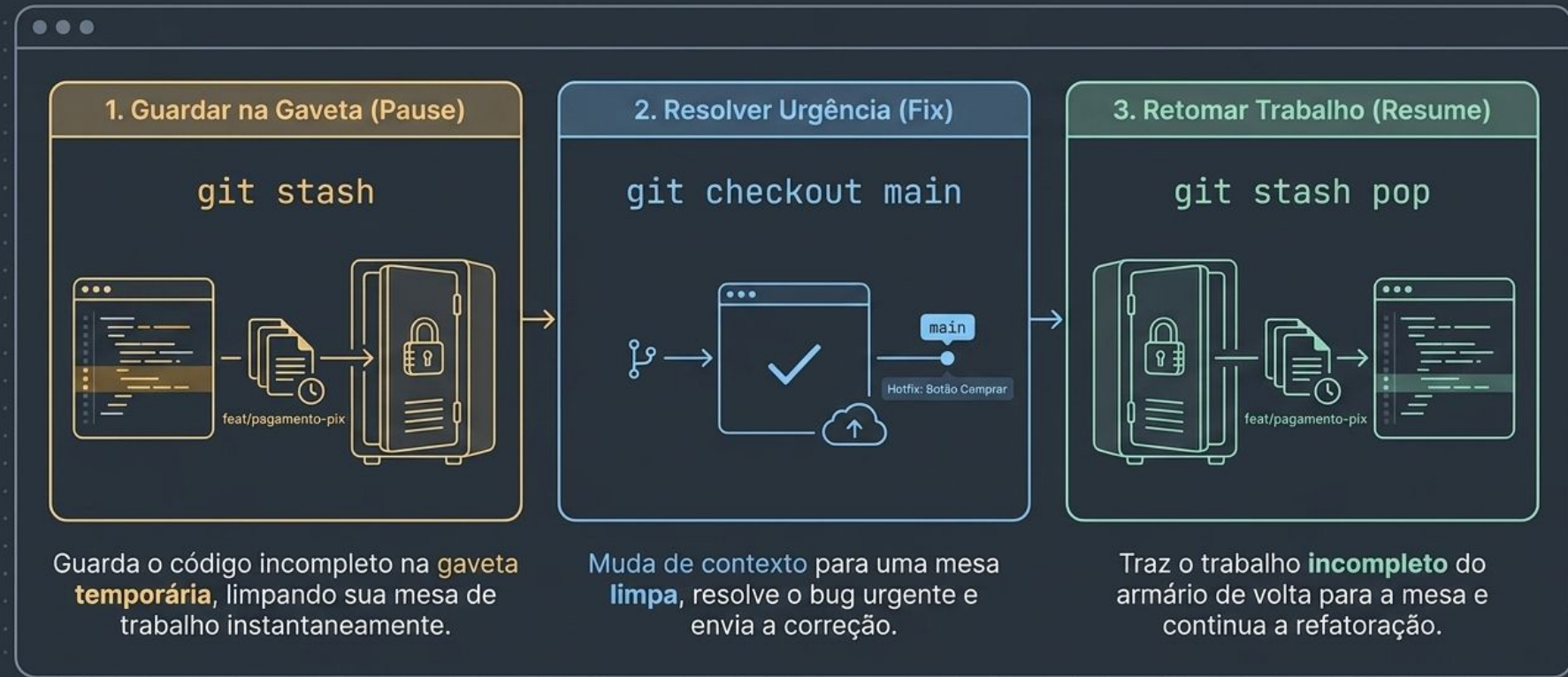
Proibido Reescrever. Apenas Avançar.



`git revert <hash>`: Em vez de apagar o passado, cria um novo commit que desfaz cirurgicamente as alterações do commit defeituoso. A única forma segura de corrigir erros em produção.

Módulo 2: O Armário Temporário e Interrupções

Cenário: Sexta-feira de Black Friday. Você está refatorando a branch `feat/pagamento-pix` quando o botão "Comprar" cai em produção. Você precisa atuar imediatamente, mas não pode commitar seu código quebrado.



A Sala de Cirurgia: Anatomia de um Conflito (Merge Conflict)

```
Terminal - git merge

<<<<<<< HEAD
codigo_botao_comprar_atual()
=====
novo_layout_botao_comprar()
>>>>>> feat/novo-layout
```

```
✓ Accept Current Change  ➡ Accept Both Changes  ↩ Accept Incoming Change

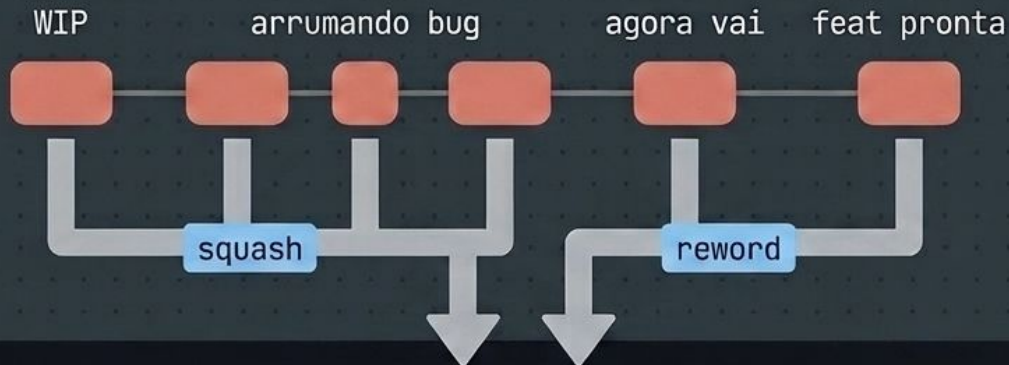
15
16 <<<<<<< HEAD (Current Change)
   ↓ codigo_botao_comprar_atual()
17 =====
18 >>>>>> feat/novo-layout (Incoming Change)
   ↑ novo_layout_botao_comprar()
19
```

Conflitos não são erros. São pausas de segurança.
O Git recusa-se a adivinhar e exige uma decisão humana clara.

Rebase Interativo: “Vista-se Bem para o Pull Request”

Antes

Ativa com o rebase -i depois usa o squash ou reword para cada commit (4 últimos no exemplos)



Depois



feat(pix):
de pagamen

```
git rebase -i HEAD~4
```

Squash: Combina vários commits em um só. Útil para juntar commits pequenos e intermediários ('WIP', 'bug fix') em um único commit coeso, mantendo o histórico limpo.

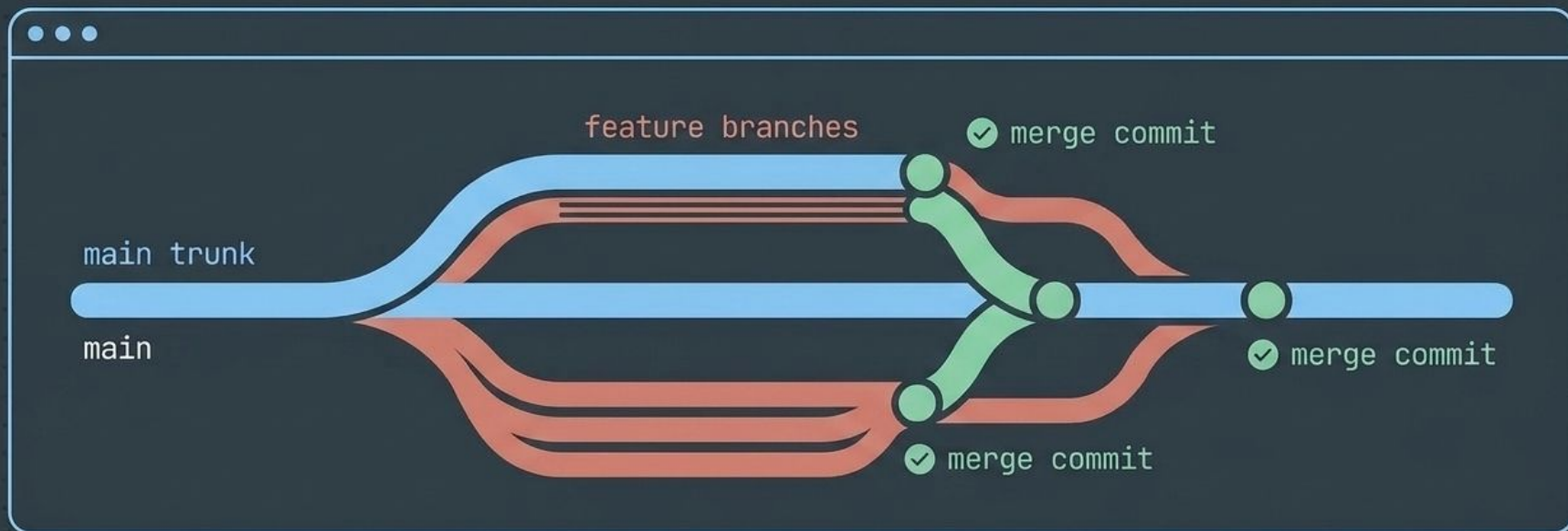
Rebase: Um comando poderoso para Reescrever a História do Git. Ele permite mover uma branch ou, no caso do modo interativo (-i), editar, combinar e reordenar commits antes de serem integrados.

Reword: Altera a mensagem de um commit. Permite corrigir erros de digitação ou melhorar a descrição de um commit sem modificar o conteúdo do código.

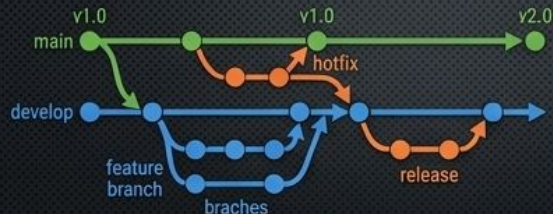
Organize a bagunça localmente antes de apresentar seu código ao mundo.

Módulo 3: Modelos de Branches Corporativas (Blueprints)

Branches no Git são apenas ponteiros móveis e extremamente leves. Criá-las é grátis e instantâneo. O verdadeiro custo está na gestão de como e quando essas linhas se encontram.



GitFlow: O Padrão Enterprise



Foco em estabilidade e cerimônia. Ideal para softwares com janelas de lançamento rígidas, onde a branch main reflete estritamente o que está em produção naquele milissegundo.

Resumo & Exemplo

Complexidade: Alta
Frequência: Agendada / Cíclica
Ideal para: Grandes Corporações

Exemplo Prático: Lançamento de uma versão principal (v2.0) de um software bancário com um ciclo de testes rigoroso.

GitHub Flow: Simplicidade e Agilidade



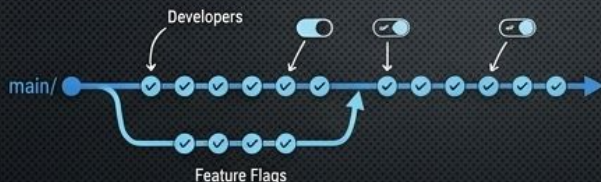
Ciclo leve baseado em Branches de Funcionalidade e Pull Requests. Ideal para deploy contínuo e equipes que valorizam a rapidez.

Resumo & Exemplo

Complexidade: Média
Frequência: Contínua (Deploy por PR)
Ideal para: Equipes Ágeis

Exemplo Prático: Desenvolvimento de uma nova funcionalidade em um site de e-commerce, com deploy imediato após aprovação do PR.

Trunk-Based Development: O Santo Graal do DevOps



Integração Extrema

Desenvolvedores integram código à linha principal várias vezes ao dia. Fim dos conflitos gigantes de fim de sprint.

Feature Flags

Código inacabado vai para a produção, mas fica oculto por chaves de ativação (Feature Toggles).

Confiança

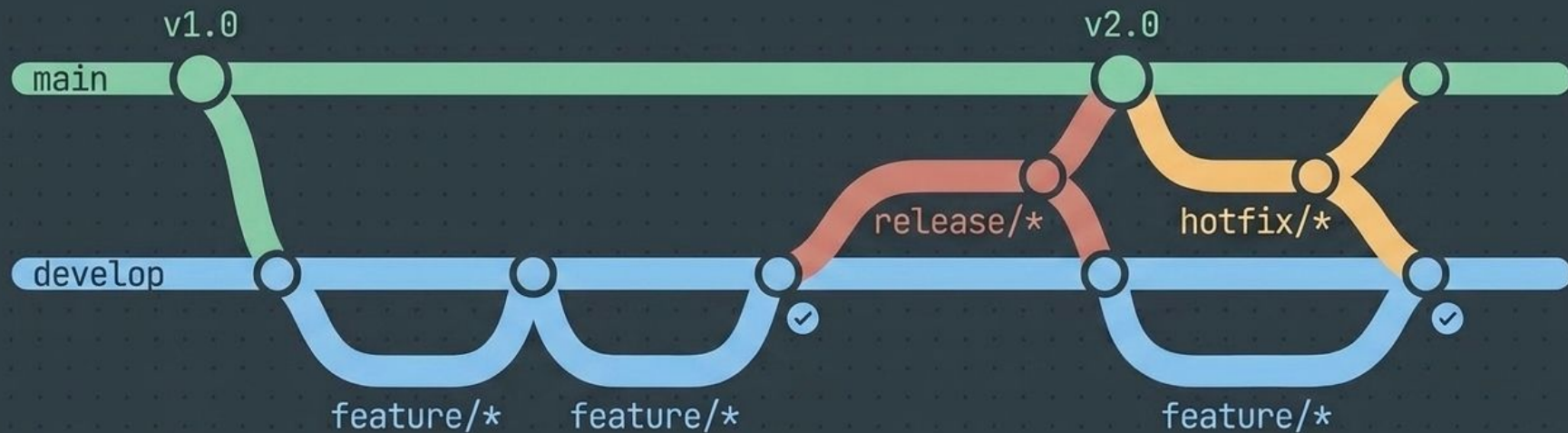
Requer uma suíte de testes automatizados implacável. Se quebrar, reverte-se imediatamente.

Resumo & Exemplo

Complexidade: Baixa (exige maturidade)
Frequência: Ultra-Rápida (Múltiplas/dia)
Ideal para: Equipes Elite DevOps

Exemplo Prático: Uma grande empresa de tecnologia onde centenas de devs commitam diretamente no trunk diariamente, usando feature flags.

GitFlow: O Padrão Enterprise



Foco em estabilidade e cerimônia. Ideal para softwares com janelas de lançamento rígidas, onde a branch **main** reflete estritamente o que está em produção naquele milissegundo.

Trunk-Based Development: O Santo Graal do DevOps



Integração Extrema

Desenvolvedores integram código à linha principal **várias vezes ao dia**. Fim dos conflitos gigantes de fim de sprint.

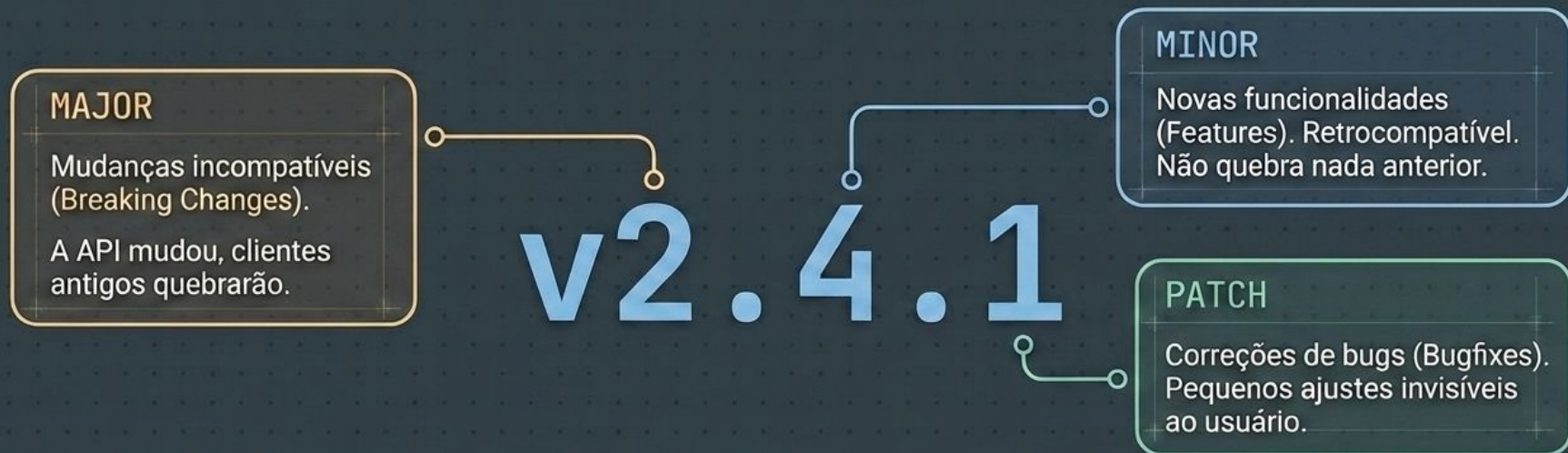
Feature Flags

Código inacabado vai para a produção, mas fica oculto por chaves de ativação (Feature Toggles).

Confiança

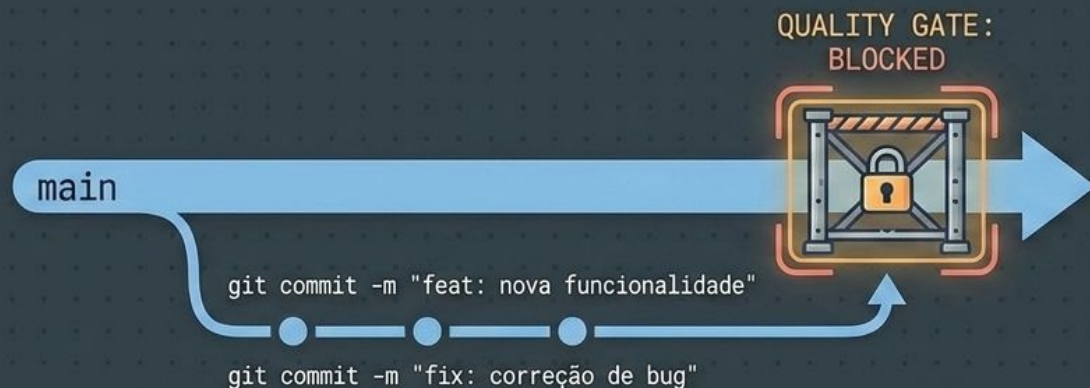
Requer uma **suíte de testes automatizados** implacável. Se quebrar, **reverte-se** imediatamente.

Módulo 4: Releases, SemVer e Automação DevOps



O Versionamento Semântico (SemVer) é um padrão que define regras claras para números de versão, comunicando o impacto das mudanças. Na Automação DevOps, ferramentas de CI/CD usam essas regras para decidir automaticamente como construir, testar e implantar o software, garantindo que atualizações seguras sejam liberadas sem intervenção humana.

Regras de Proteção e Pull Requests (PRs)



Requisitos para Merge

- ✓ Aprovação de 2 revisores obrigatória (Code Review).
- ✓ Pipeline de Testes Automatizados passou.
- ✓ Histórico linear (Rebase exigido).
- ✓ Autenticação forte via Chaves SSH.

A branch main é o cofre da empresa. Ninguém escreve nela diretamente. O Pull Request não é apenas um pedido de mesclagem; é a fronteira implacável de qualidade do código.

O Gatilho do DevOps: GitOps e CI/CD

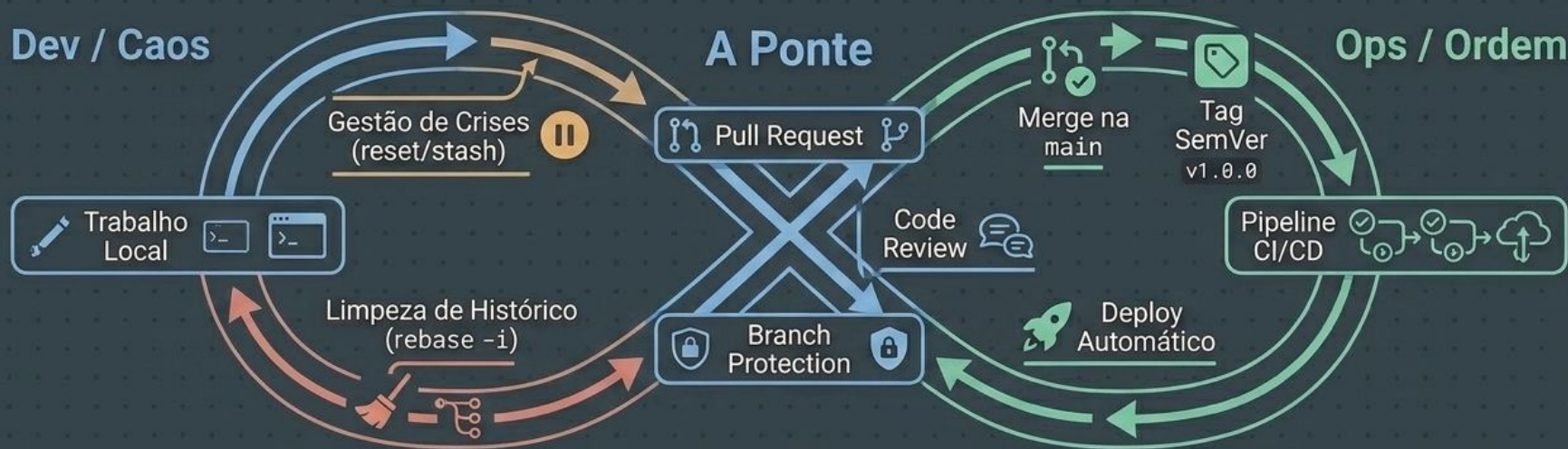


O desenvolvedor
envia o código.

O GitHub Actions
intercepta o evento.

O Git evoluiu. Ele deixou de ser apenas um repositório passivo e se tornou o gatilho ativo que constrói, testa e entrega software na nuvem.

O Ciclo de Vida de uma Alteração de Código



Do caos criativo no seu computador até a estabilidade cirúrgica do deploy em nuvem.
O Git é o motor contínuo que alimenta o software corporativo moderno.
O medo é substituído pelo controle absoluto.

Git Atividade

Trabalhar na
entrega do projeto.